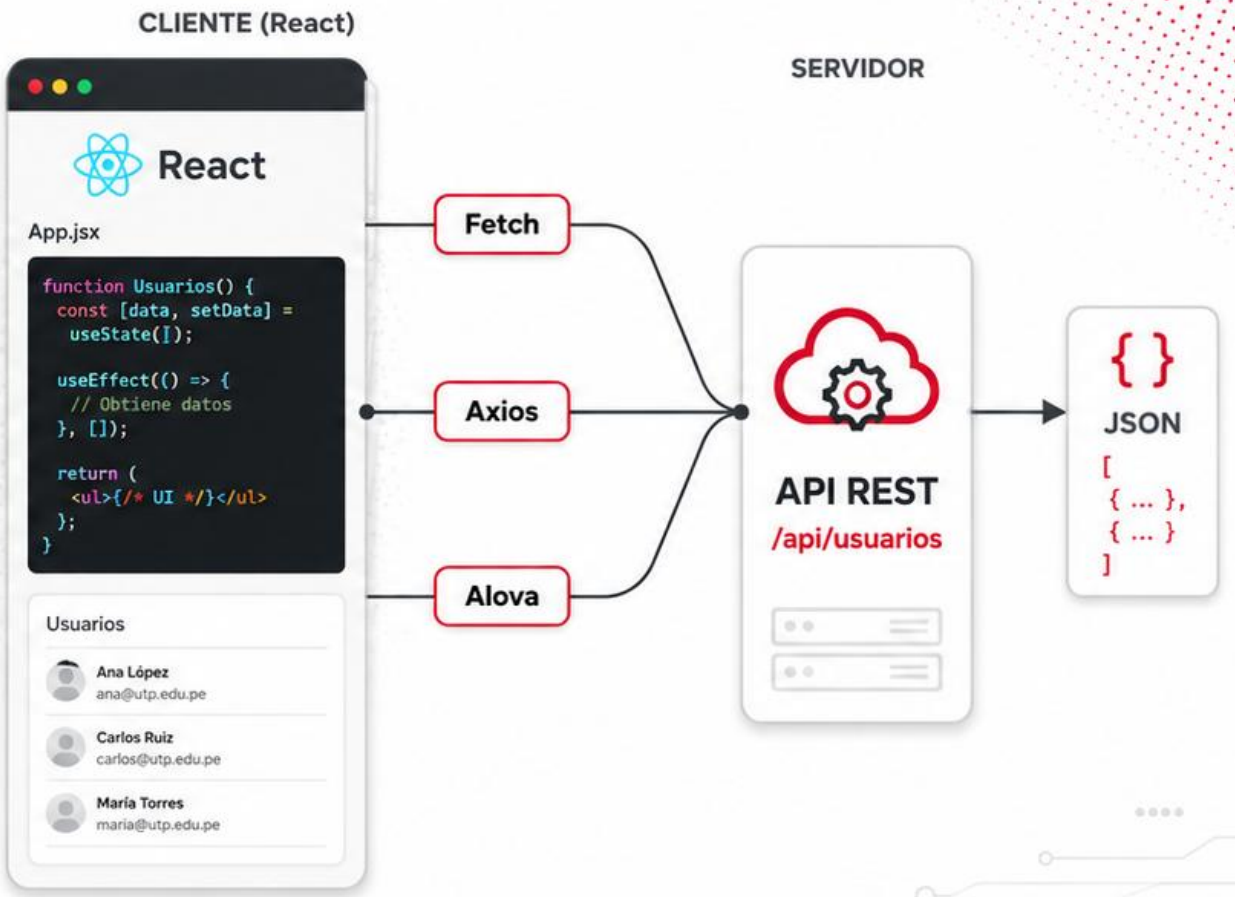
Universidad Tecnológica del Perú

React conectado al mundo.

Servicios REST con Fetch API, Axios y Alova.

Curso: JavaScript Avanzado | **Unidad 3:** React | **Semana:** 14

Servicios: en React | **Docente:** Mg. Ivan Eduardo Anchante Reyes.

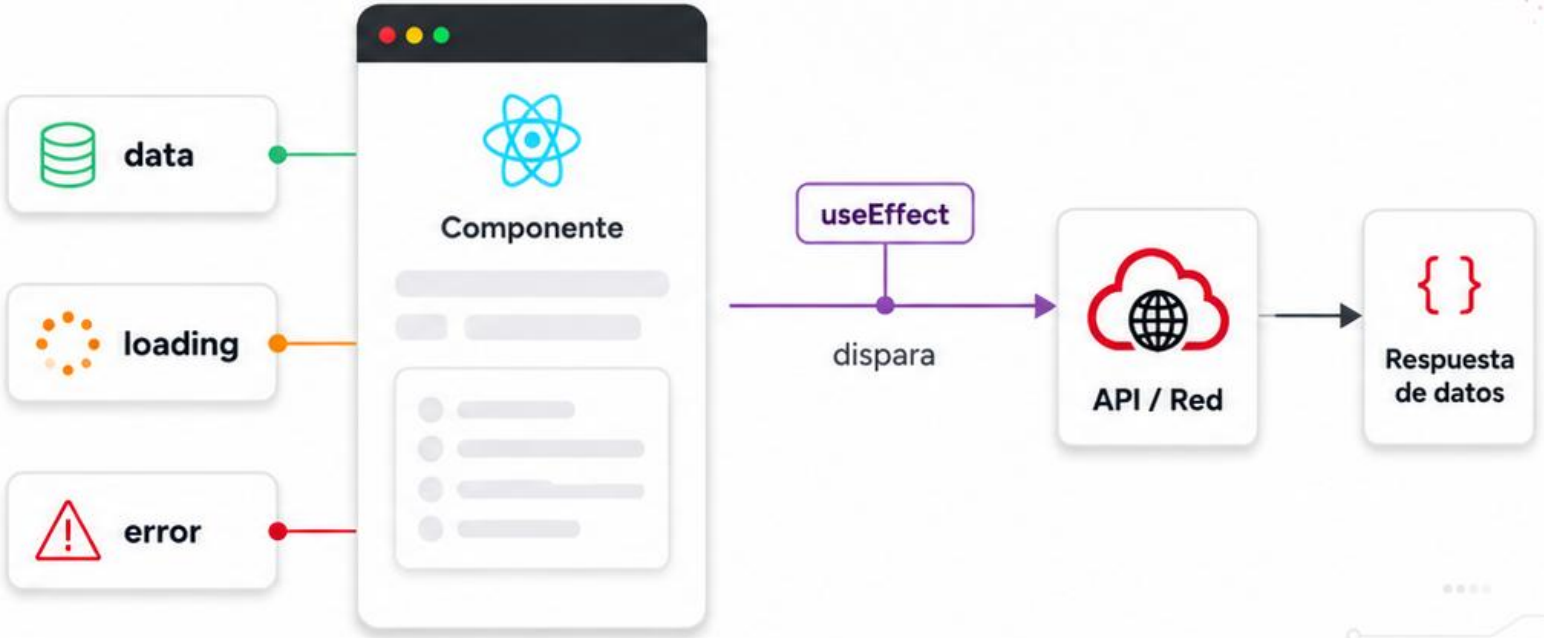


Dos Hooks que ya tenemos.

useState guarda; useEffect sincroniza

Recuperación: useState conserva data/loading/error. **useEffect** puede iniciar la carga al montar el componente.

Pregunta: ¿qué estado necesitarías para una lista que llega por red?



Universidad Tecnológica del Perú

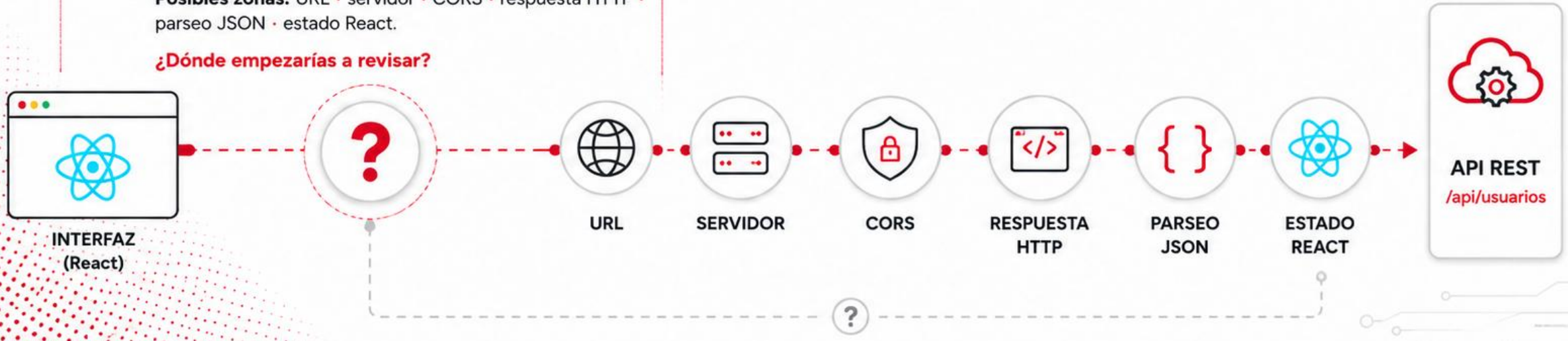
La interfaz puede estar bien... y aun así fallar.

El problema puede estar fuera del componente

Escenario: el botón funciona, el componente renderiza, pero la lista no aparece.

Posibles zonas: URL · servidor · CORS · respuesta HTTP · parseo JSON · estado React.

¿Dónde empezarías a revisar?



Universidad Tecnológica del Perú

De Hooks a datos reales.

Dudas de la semana anterior antes de avanzar

Pregunta al grupo: ¿qué duda quedó sobre **useState, useEffect, useRef** o **Context** que podría dificultar consumir una API?

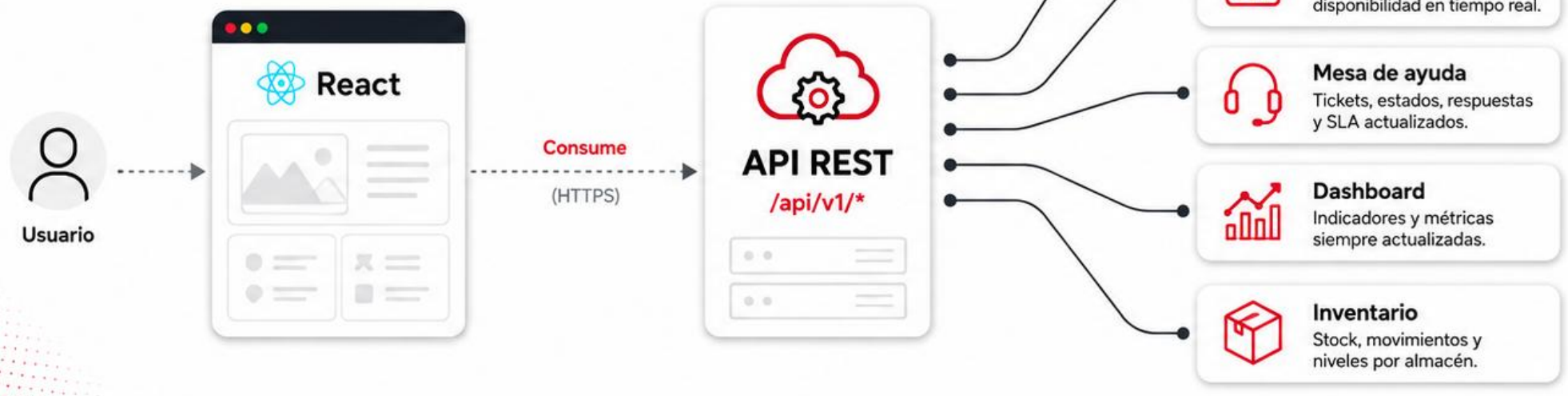
Seleccionar **1–2 dudas** realmente relacionadas con la sesión.



Universidad Tecnológica del Perú

¿Para qué consumir una API desde React?

La interfaz adquiere datos que cambian fuera del navegador



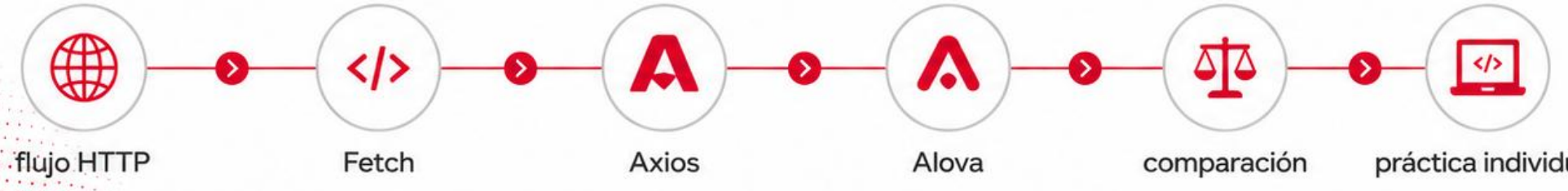
Situaciones: catálogo · reservas · mesa de ayuda · dashboard · inventario.

Pregunta: ¿qué valor aporta separar la interfaz de la fuente de datos?

Logro y ruta de la sesión

Comprender el flujo, comparar clientes y transferir.

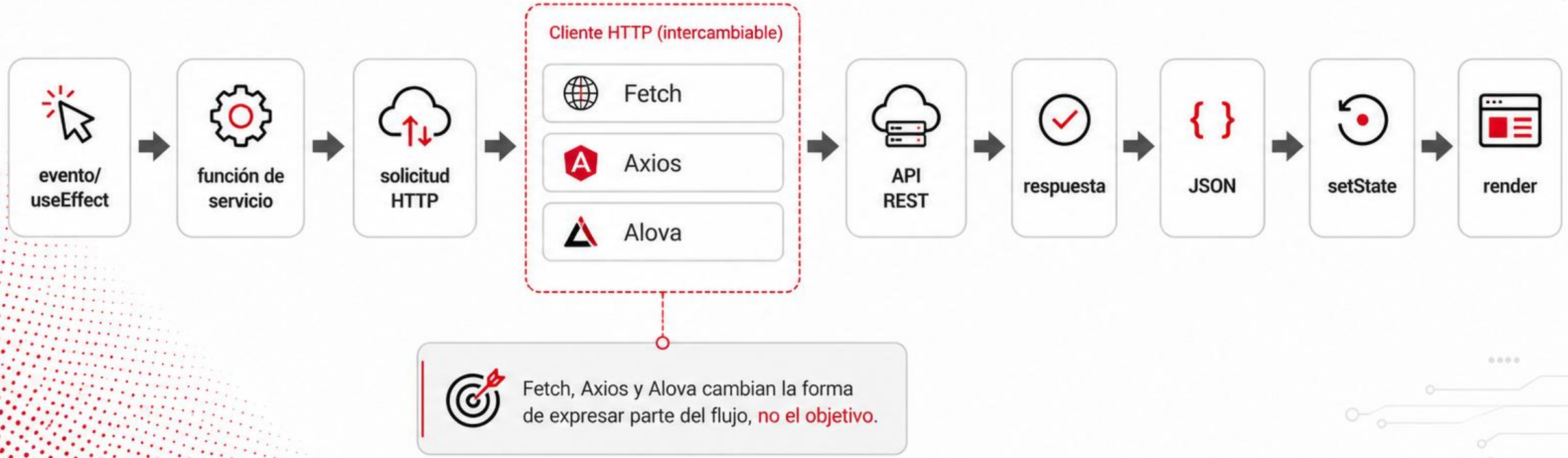
Logro: Al finalizar la sesión, el estudiante consume servicios REST desde React, diferenciando Fetch API, Axios y Alova, e implementa individualmente una interfaz que consulta y registra datos con un cliente HTTP seleccionado, manejando carga y error.



Universidad Tecnológica del Perú

El flujo es el mismo aunque cambie la herramienta

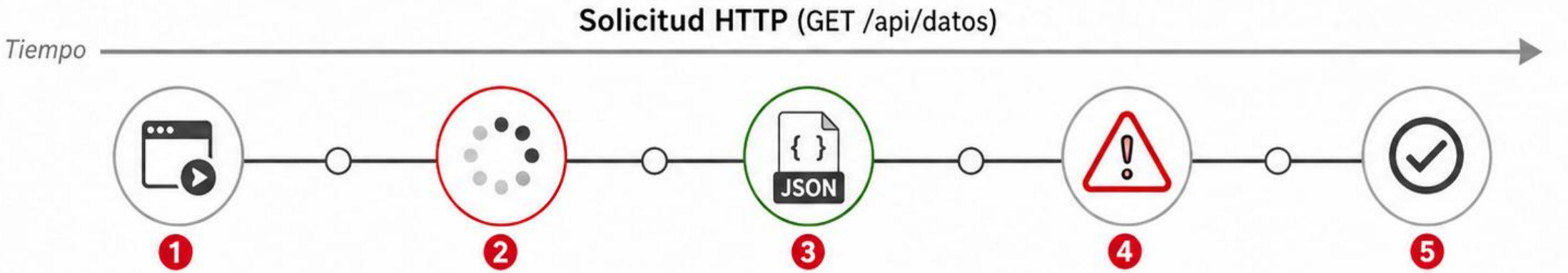
React pide; la API responde; el estado cambia



Universidad Tecnológica del Perú

Tres estados mínimos de una petición

loading • data • error



	1	2	3	4	5
loading	false	true	true	true	false
data	[]	[]	{ ... }	[]	{ ... } (datos disponibles)
error	""	""	""	"Error en la petición"	""

HTTP: lo mínimo que debes leer

Método, endpoint y estado

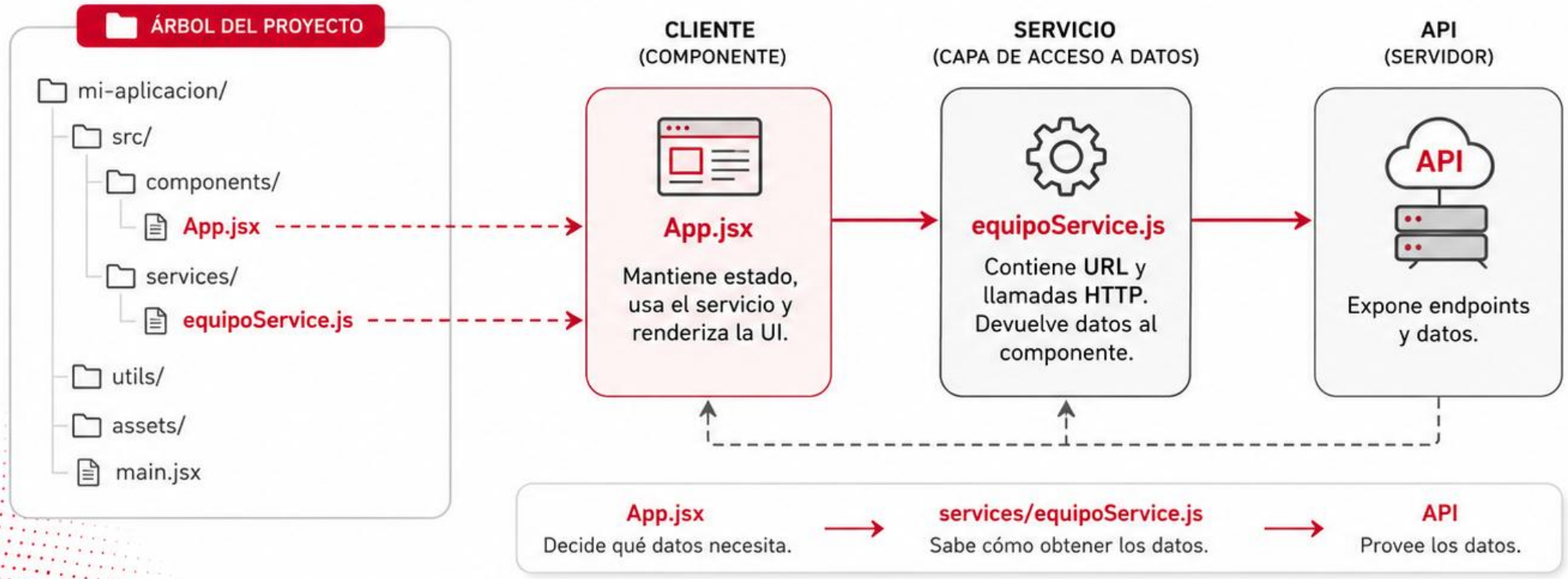
GET consulta · **POST** crea

Estados frecuentes en el laboratorio:
200 OK · **201** Created · **400** Bad Request · **404** Not Found · **500** Server Error

Method	URL	Status	Response
GET	/equipos	200 OK	Lista de equipos.
GET	/equipos/1	200 OK	Detalle del equipo con id = 1.
POST	/equipos	201 Created	Equipo creado correctamente.
POST	/equipos	400 Bad Request	Datos inválidos o incompletos.
GET	/equipos/999	404 Not Found	El equipo solicitado no existe.
POST	/equipos	500 Server Error	Error interno del servidor.

Separa el acceso a datos del componente

El componente decide qué necesita; el servicio conoce cómo pedirlo.



El **módulo de servicio** contiene URL y llamadas HTTP; el **componente** mantiene estado y renderiza.

Fetch API: el cliente nativo del navegador

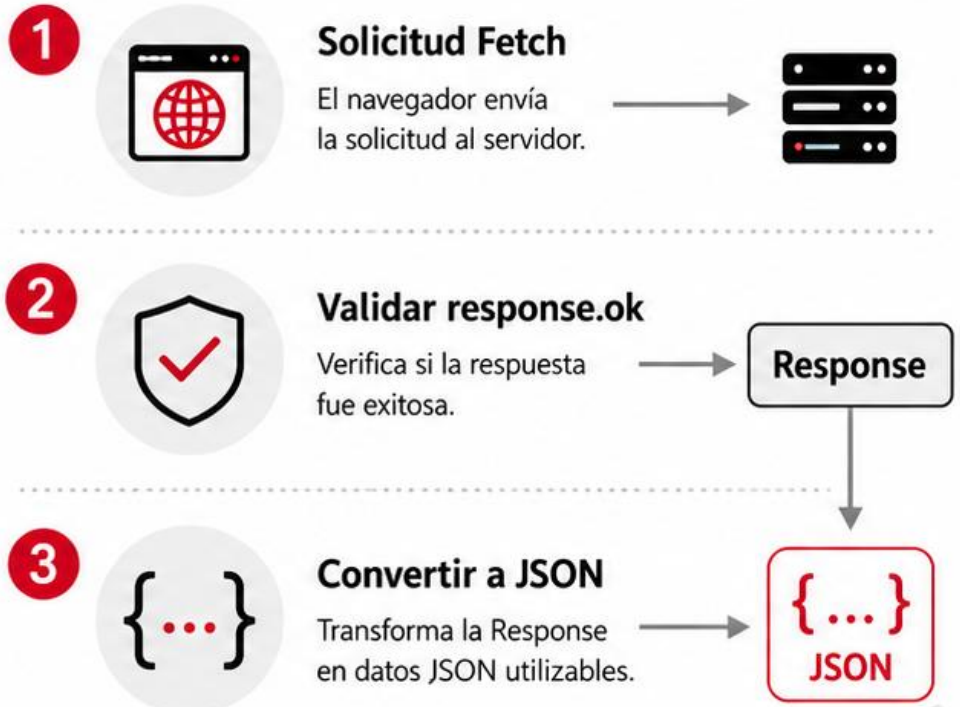
Sin paquete adicional para hacer la solicitud

- **fetch(url)** devuelve una **Promise** con **Response**.
- Para nuestro laboratorio: **await fetch(...)**, comprobar **response.ok** y luego **await response.json()**.

```
const response = await fetch(url);

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const data = await response.json();
```



Predicción: ¿404 entra al catch?

Con Fetch, un error HTTP requiere comprobación explícita

- **Código:** `const r = await fetch(url); console.log(r.ok, r.status);`
- **Pregunta:** si el servidor responde 404, ¿la Promise se rechaza por ese motivo? **Responde antes** de mostrar el resultado.

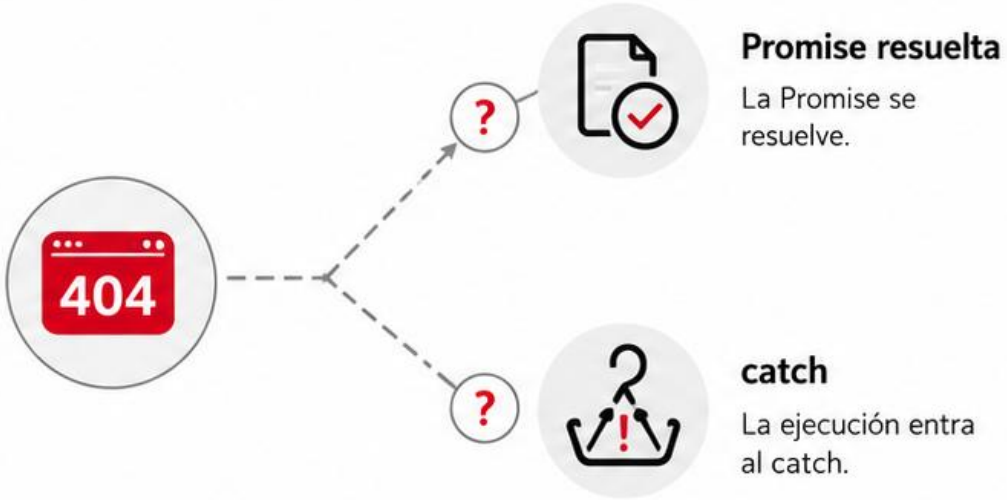
```
const r = await fetch(url);

console.log(r.ok, r.status);

// 404: la Promise puede resolverse;
// r.ok será false.
```

Tu predicción

Si el servidor responde 404...



?

¿Cuál camino crees que tomará Fetch con un 404?

Fetch GET: listar equipos

Solicitud, validación y JSON

```
export async function listarEquipos() {

  const r = await fetch(`${API}/equipos`);

  if (!r.ok) throw new Error(`HTTP ${r.status}`);

  return r.json();
}
```



Código visible: función **listarEquipos()** que hace **GET** a **/equipos**, valida **response.ok** y retorna **response.json()**.

Fetch POST: enviar JSON

method + headers + body

```
export async function crearEquipo(equipo) {

  const r = await fetch(`${API}/equipos`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(equipo)
  });

  if (!r.ok) throw new Error(`HTTP ${r.status}`);

  return r.json();
}
```

objeto JS
El objeto equipo en JavaScript.

JSON.stringify(equipo)
Convierte el objeto a JSON para el cuerpo de la petición.

POST /equipos
Envío con method: 'POST' y Content-Type: 'application/json' en headers y el JSON en body.

201
La API confirma la creación con estado 201 (Created).

objeto creado
Se valida **response.ok** y se devuelve el JSON creado.

Código visible:
POST /equipos con **Content-Type application/json** y **body: JSON.stringify(equipo).**

Validar **response.ok** y devolver JSON creado.

Universidad Tecnológica del Perú

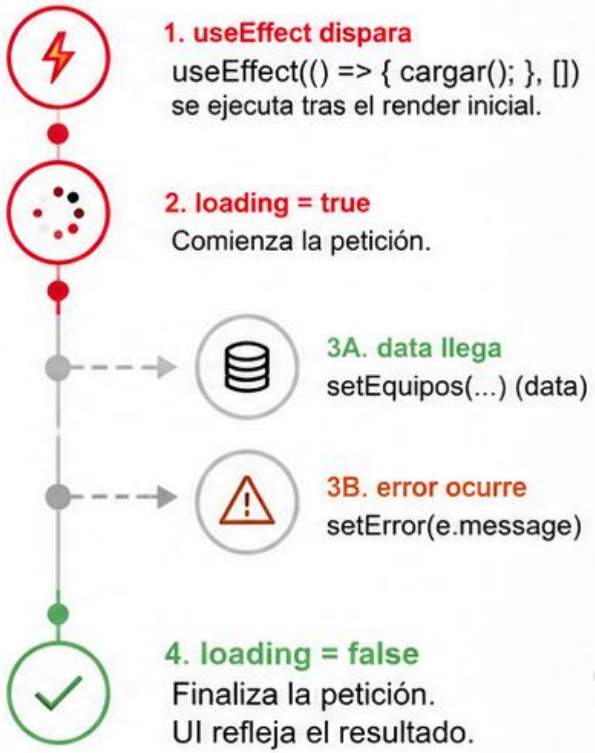
React coordina la petición

useEffect dispara; el estado representa el resultado

- **Patrón del laboratorio:** useEffect(() => { cargar(); }, []); cargar() usa try/catch/finally; setLoading(true), setData(...), setError(...), setLoading(false).

```
useEffect(() => { cargar(); }, []);

async function cargar() {
  setLoading(true);
  try { setEquipos(await listarEquipos()); }
  catch (e) { setError(e.message);  }
  finally { setLoading(false); }
}
```



Universidad Tecnológica del Perú

Demostración Fetch completa

InventarioLab: GET + POST + estados de UI

Secuencia de demo: **1** iniciar backend; **2** cargar lista; **3** registrar equipo; **4** observar Network; **5** detener backend y recargar; **6** restaurar servidor.

1 Iniciar backend

```
$ cd backend
$ npm install
$ npm run dev

> inventariolab-backend@1.0.0 dev
> nodemon server.js

Servidor corriendo en http://localhost:3001
API lista para recibir peticiones.
```

2 Cargar lista (GET)

InventarioLab — Actualizando...

Equipos (4)

ID	Nombre	Tipo	Estado
1	Microscopio	Óptico	Activo
2	Centrífuga	Laboratorio	Activo
3	Balanza analítica	Precisión	Activo
4	Espectrofotómetro	Óptico	Activo

3 Registrar equipo (POST)

Registrar nuevo equipo

Nombre: Termómetro digital

Tipo: Medición — Estado: Activo

Guardar

Equipo registrado ID: 5

4 Observar Network

Name	Method	Status	Type	Time
equipos	GET	200	json	212 ms
equipos	POST	201	json	135 ms
equipos	GET	200	json	188 ms

5 Detener backend y recargar

InventarioLab — Sin conexión

No se pudo conectar con el servidor

Asegúrate de que el backend esté ejecutándose.

Reintentar

6 Restaurar servidor y recuperar datos

InventarioLab — Conectado

Equipos (5)

ID	Nombre	Tipo	Estado
1	Microscopio	Óptico	Activo
...	...	...	...
5	Termómetro digital	Medición	Activo

Universidad Tecnológica del Perú

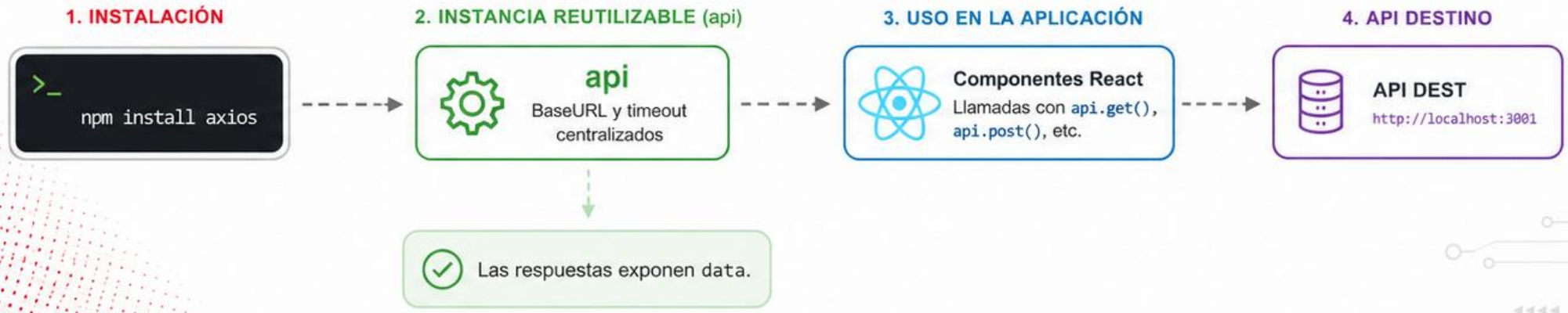
Axios: una API de cliente más directa

Instalación y una instancia reutilizable

- **Instalación:** `npm install axios`.
- **Idea:** `axios.create({ baseURL, timeout })` centraliza configuración.
- Las respuestas exponen `data`.

```
import axios from 'axios';

const api = axios.create({
  baseURL: '[http://localhost:3001](http://localhost:3001)',
  timeout: 5000
});
```



Axios GET: response.data

Menos pasos visibles para obtener el cuerpo

Fetch GET (referencia)

1. Solicitar
fetch('/equipos')

2. Verificar estado
if (!res.ok) throw new Error(...)

3. Obtener JSON
const data = await res.json()

→ **Más pasos visibles**

Axios GET

```
export async function listarEquipos() {
  const { data } = await api.get('/equipos');
  return data;
}
```

Axios expone el cuerpo directamente en **response.data**.

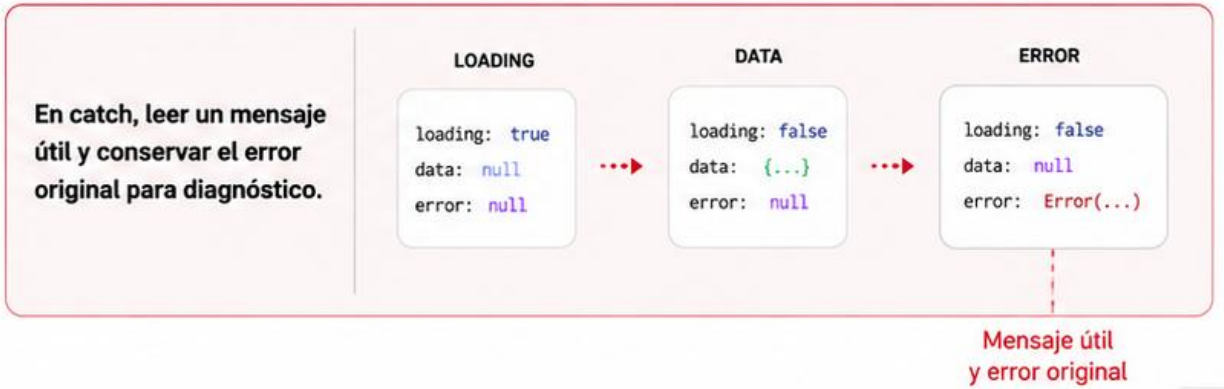
Mismo resultado
Array de equipos (JSON)
`[ { ... }, { ... }, ... ]`

Axios POST y errores

Enviar objeto sin serializarlo manualmente en este caso.

```
1  export async function crearEquipo(equipo) {
2
3    const { data } = await api.post('/equipos', equipo);
4    return data;
5  }
```





Universidad Tecnológica del Perú

Demostración Axios

Misma app, otro cliente HTTP

Comparar únicamente el módulo de servicio con la versión Fetch.

LA APP NO CAMBIA (App.jsx)

App.jsx

Gestión de Equipos

Lista de equipos

ID	Nombre	Categoría	Acciones
1	Laptop Lenovo	Tecnología	
2	Proyector Epson	Audiovisual	
3	Impresora HP	Oficina	

Registrar nuevo equipo

Nombre del equipo | Categoría | Registrar

=

LO QUE CAMBIA: MÓDULO DE SERVICIO

equipoServiceFetch.js

```
// Obtener lista
export const listarEquipos = async () => {
  const res = await fetch('/api/equipos');
  if (!res.ok) throw new Error('Error al listar');
  return res.json();
};

// Registrar
export const registrarEquipo = async (equipo) => {
  const res = await fetch('/api/equipos', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringfiy(equipo)
  });
  if (!res.ok) throw new Error('Error al registrar');
  return res.json();
};
```

↔

equipoServiceAxios.js

```
// Obtener lista
import axios from 'axios';
const api = axios.create({ baseURL: '/api' });

export const listarEquipos = async () => {
  const { data } = await api.get('/equipos');
  return data;
};

// Registrar
export const registrarEquipo = async (equipo) => {
  const { data } = await api.post('/equipos', equipo);
  return data;
};
```

La app (UI y flujo) se mantiene igual.
Solo cambiamos el módulo de servicio.

SECUENCIA DE LA DEMO

1. **Cargar lista** — Ver equipos existentes.
2. **Registrar** — Agregar un nuevo equipo.
3. **Revisar Network** — Ver la petición HTTP (método, URL, estado).
4. **Provocar error** — Forzar un error (ej. 500) y validar manejo.

Fetch vs Axios

No existe ganador universal

Fetch: integrado en navegador, sin dependencia, exige manejo explícito de Response. **Axios:** dependencia, instancia/configuración cómoda, data directa.

Criterio	Fetch	Axios
dependencia	✓ **Sin dependencia** Integrado en el navegador	✗ **Con dependencia** Requiere instalación
configuración	– **Configuración mínima** Uso directo y simple	✓ **Instancia y configuración cómoda** Defaults, interceptores, baseURL, etc.
control de Response	– **Manejo explícito** response.json(), status, ok, headers	✓ **Abstracción de Response** data directo, error handling integrado
ergonomía	– **Más verboso** Mayor código para tareas comunes	✓ **Más ergonómico** Menos código, API intuitiva

La decisión depende de restricciones y equipo.

Alova v3: instancia + adaptador

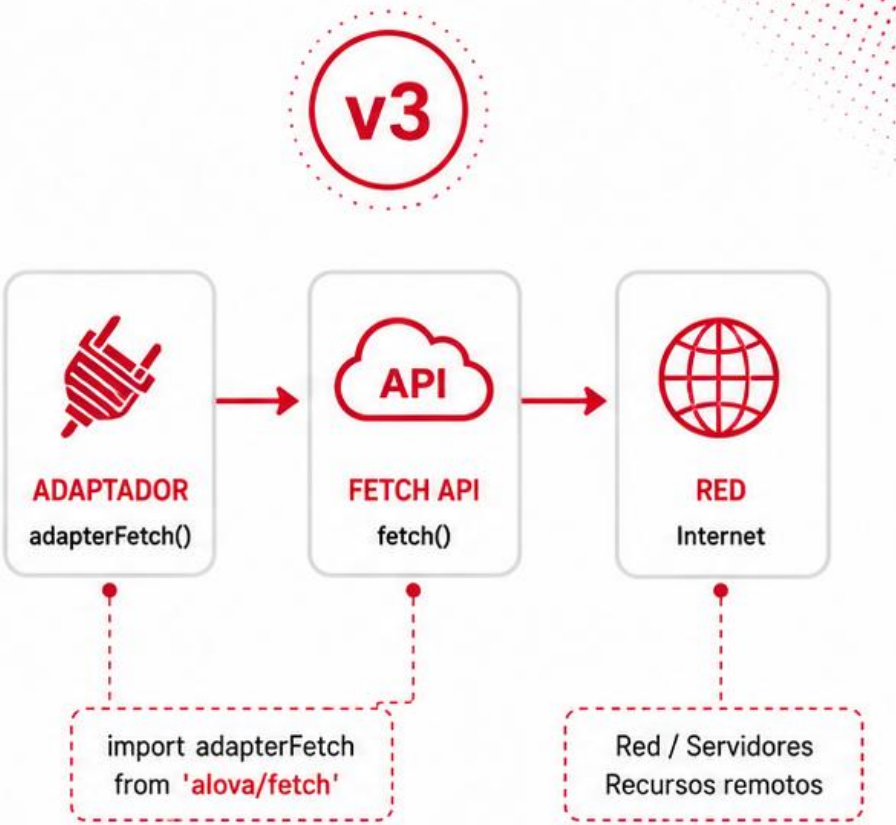
Configuración explícita sobre un adaptador Fetch

```
import { createAlova } from 'alova';

import adapterFetch from 'alova/fetch';

const api = createAlova({
  baseURL: '[http://localhost:3001](http://localhost:3001)',
  requestAdapter: adapterFetch(),
  responded: response => response.json()
});
```

Importar **createAlova** desde **alova** y **adapterFetch** desde **alova/fetch**.



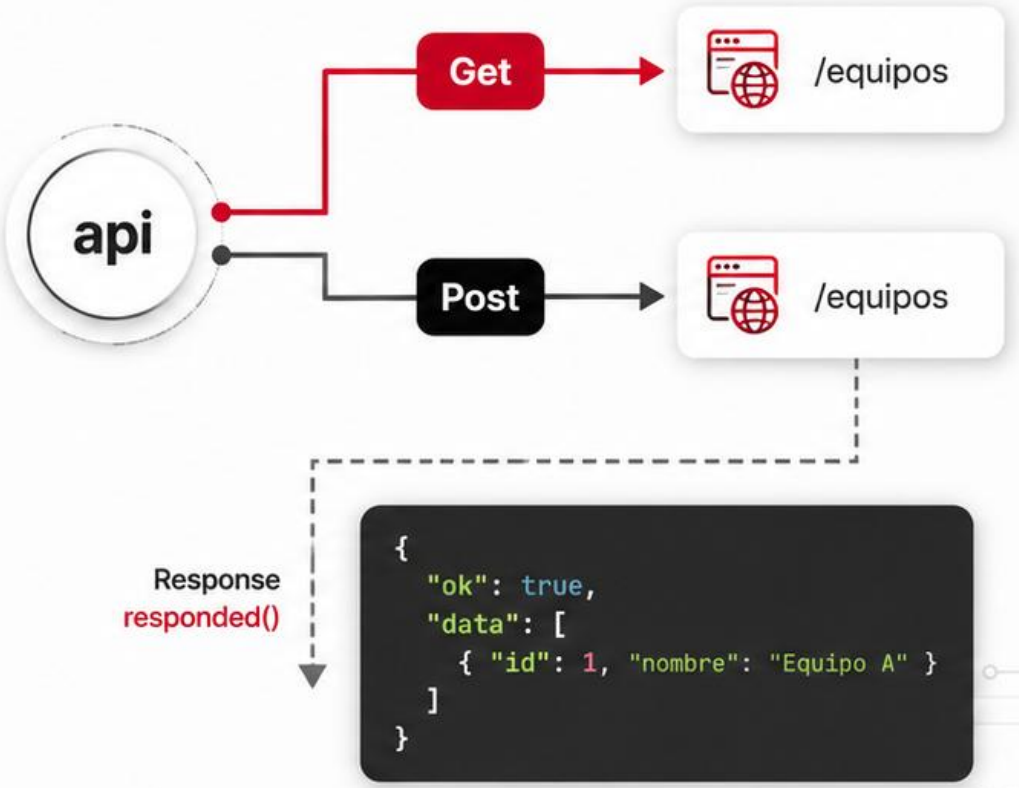
Alova GET y POST

Métodos creados desde la instancia

Código visible: await api.Get("/equipos") y await api.Post("/equipos", equipo).
La función **responded** transforma la Response a JSON para este laboratorio.

```
export const listarEquipos = () =>
   api.Get('/equipos');

export const crearEquipo = equipo =>
   api.Post('/equipos', equipo);
```

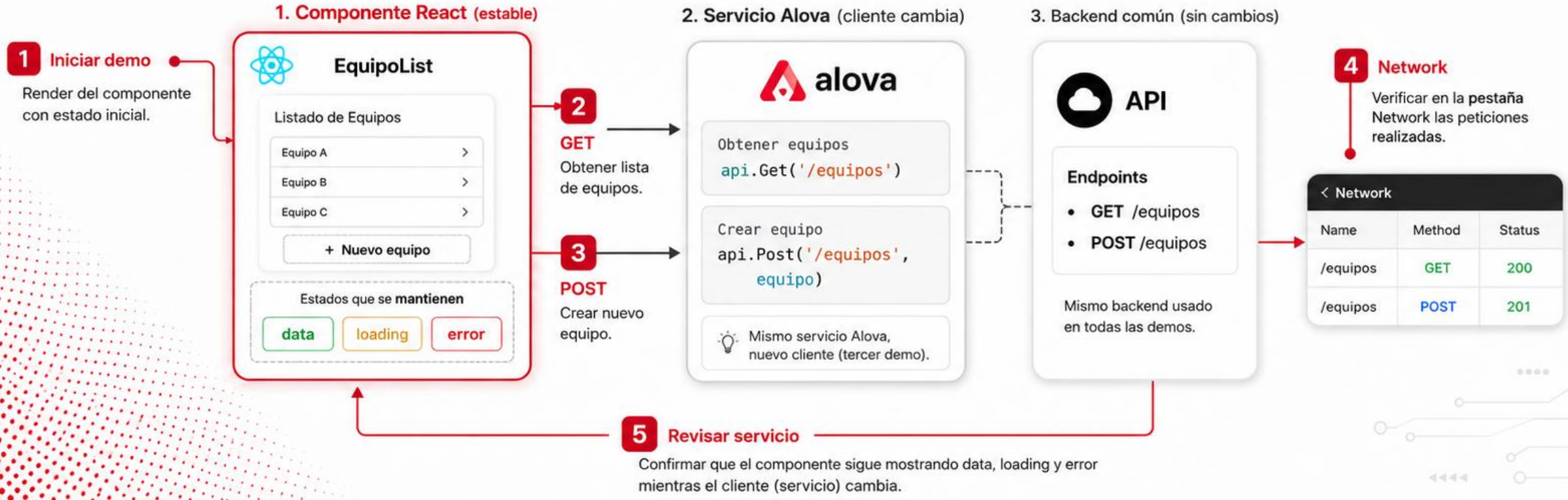


Universidad Tecnológica del Perú

Demostración Alova

Mismo backend, tercer cliente

En esta demo cambiamos el cliente (servicio) pero el componente mantiene **data**, **loading** y **error** igual que en los otros demos.



Universidad Tecnológica del Perú

Tres clientes, una decisión técnica

Compara antes de elegir

Cliente	Dependencia	Legibilidad del equipo	Configuración compartida	Extensibilidad
Fetch nativo / control explícito.	Muy baja (nativo)	Media (API básica)	Baja (manual)	Baja (poco extendible)
Axios cliente de propósito general / configuración cómoda.	Media (librería ligera)	Alta (API clara)	Alta (interceptors, config global)	Media (plugins básicos)
Alova instancia y métodos sobre adaptadores / capacidades adicionales.	Media (librería ligera)	Media (curva moderada)	Alta (instancias, config por equipo)	Alta (métodos, adaptadores y life cycle)

No existe ganador universal
Elige según el contexto y las prioridades de tu equipo.

Universidad Tecnológica del Perú

Diagnóstico por capas

Network primero; después código

1. confirmar backend /health
2. revisar URL y método
3. revisar Network y status
4. revisar CORS
5. revisar transformación JSON/data
6. revisar setState/render

Backend

```
$ curl http://localhost:3001/health
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "ok",
  "time": "2025-05-12T10:30:00Z"
}
$
```

Solicitud

GET http://localhost:3001/api/equipos

Headers

```
Accept: application/json
Content-Type: application/json
```

Body

—

Network

Name	Status	Type	Time
equipos	200	fetch	120 ms

```
200 OK
Content-Type: application/json
Access-Control-Allow-Origin: *

Response Body
{ "ok": true, "data": [...] }
```

Servicio (CORS)

- ✓ Origin permitido
- ✓ Métodos permitidos (GET, POST)
- ✓ Headers permitidos
- ✗ CORS no configurado correctamente

⇒ La respuesta será bloqueada por el navegador.

Transformación

```
{
  "ok": true,
  "data": [
    { "id": 1, "nombre": "Equipo A" },
    { "id": 2, "nombre": "Equipo B" }
  ]
}
```

Transformación

```
const equipos = data.map(e => ({
  id: e.id,
  nombre: e.nombre
}));
```

Componente (Render)

```
const [equipos, setEquipos] =
  useState([]);

useEffect(() => {
  fetch('/api/equipos')
    .then(res => res.json())
    .then(data => setEquipos(data))
    .catch(err => console.error(err));
}, []);

return (
  <EquiposTable data={equipos} />
);
```

⚠ **CORS no se corrige cambiando JSX.**

Universidad Tecnológica del Perú

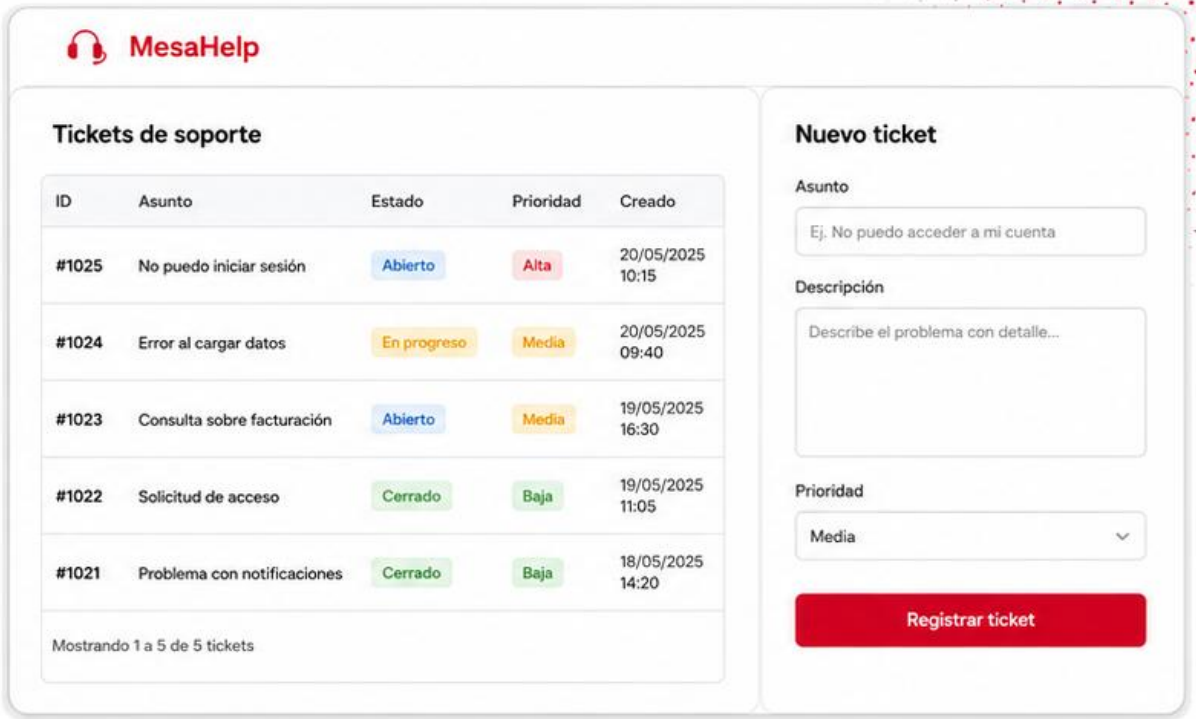
Reto individual: MesaHelp React

Consume un servicio REST en un contexto nuevo.

Modalidad: INDIVIDUAL

Tiempo: 55 minutos

Producto: interfaz React funcional para listar y registrar tickets de soporte usando UN cliente HTTP: Fetch, Axios o Alova.



Elige tu cliente HTTP:

Fetch ○ Axios ○ Alova ○

Caso: MesaHelp

Los laboratorios necesitan registrar incidencias.

Contexto del caso

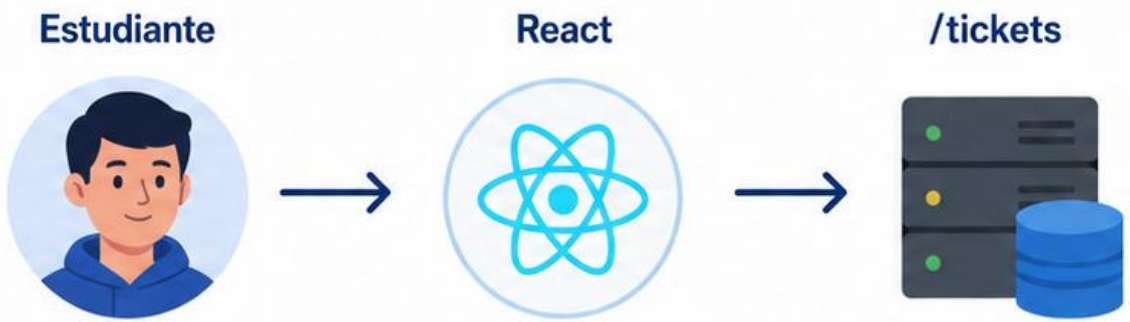
MesaHelp es un sistema para que los estudiantes reporten incidencias en los laboratorios. La interfaz permite visualizar tickets existentes y registrar nuevos.

Estructura de cada ticket

- **id:** identificador único (número)
- **titulo:** descripción breve de la incidencia (texto)
- **area:** laboratorio o área afectada (texto)
- **prioridad:** nivel de prioridad (*baja, media, alta*)
- **estado:** situación actual (*abierto, en progreso, cerrado*)

Contrato de la API

- **GET /tickets** → Obtener todos los tickets
- **POST /tickets** → Registrar un nuevo ticket



Ejemplo de ticket (JSON)

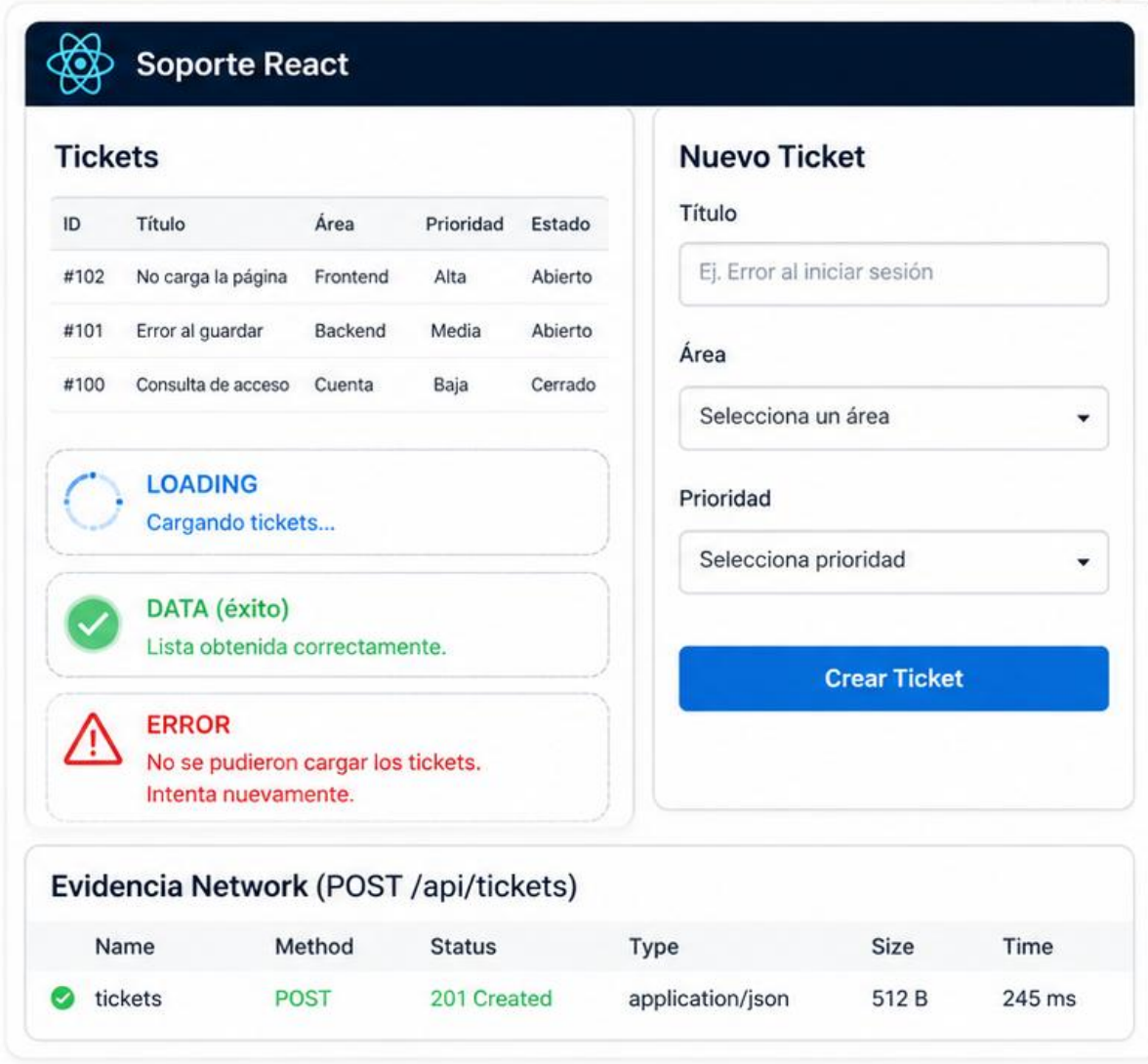
```
{
  "id": 125,
  "titulo": "No funciona el proyector",
  "area": "Laboratorio 2",
  "prioridad": "alta",
  "estado": "abierto"
}
```

Cada ticket contiene: id, titulo, area, prioridad y estado. API local: GET /tickets y POST /tickets. La interfaz debe cargar tickets existentes y registrar uno nuevo.

Producto esperado

Lo que debe funcionar antes de entregar

- 1) lista visible obtenida por GET
- 2) formulario con titulo, area y prioridad
- 3) POST crea ticket y actualiza la lista
- 4) loading visible
- 5) error visible
- 6) servicio separado del componente
- 7) evidencia Network
- 8) justificación breve del cliente elegido



Elige un cliente y justifica

Una sola herramienta, una razón técnica

Opciones: Fetch, Axios o Alova. Escribe 2–4 líneas: requisito/contexto → criterio → elección. Ejemplos de criterios: no agregar dependencia, configuración compartida, coherencia con stack del equipo, abstracción preferida.

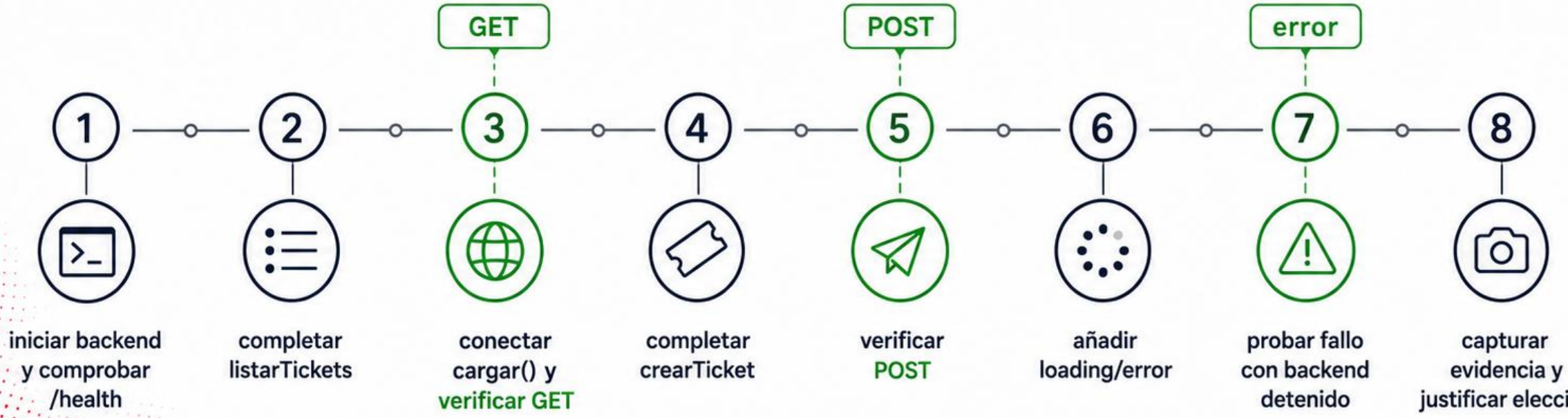


Mi elección:

_____ porque _____

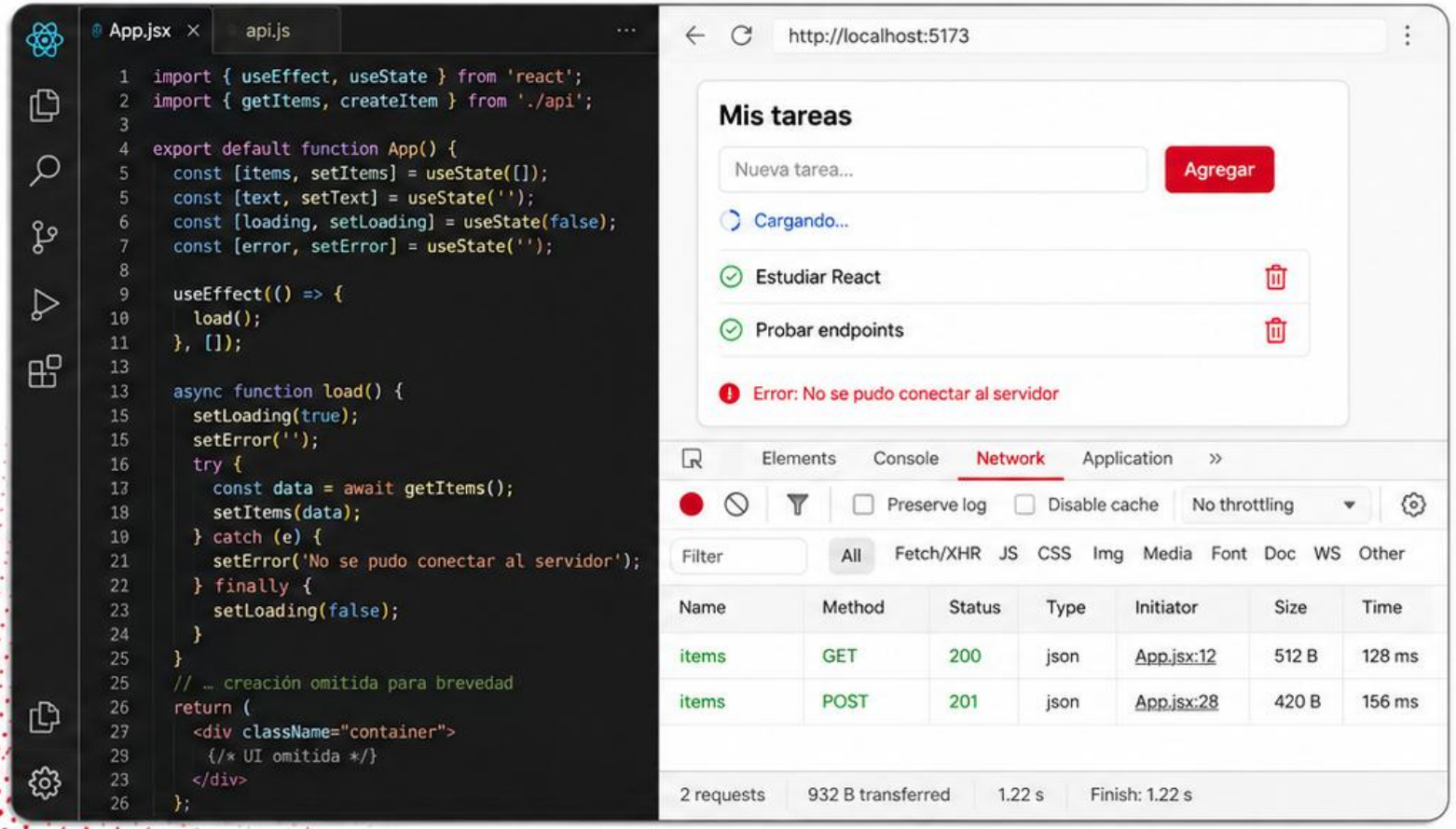
Orden de trabajo recomendado

Primero evidencia de red, después pulido



Trabajo autónomo y verificación

Construye, prueba, corrige y explica





Durante 30 minutos:
implementación individual.

Antes de entregar:

- [x] GET 200
- [x] POST 201
- [x] loading visible
- [x] error visible con servidor detenido
- [x] servicio separado
- [x] cliente consistente
- [x] justificación escrita

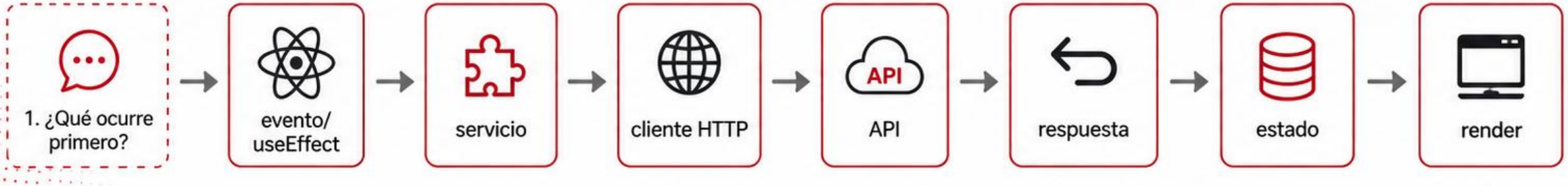
Docente:
monitorea y retroalimenta sin resolver.

Reconstruimos el flujo

Sin mirar: de React a la API y de vuelta

Pide 3 aportes para reconstruir

¿Cuál es el siguiente paso después de...?



Flujo completo (síntesis revelada)

Luego mostrar *la síntesis completa.*

Ticket de salida

Elige, justifica y diagnostica

1



Caso A: proyecto pequeño sin dependencias nuevas. ¿Qué cliente considerarías primero y por qué?

2



Caso B: GET /tickets muestra 404 en Network. ¿Qué revisarías antes del JSX?

3



Caso C: con Fetch recibes 500; ¿qué propiedad debes comprobar?

No revelar respuestas inicialmente.

Hoy conectamos; la próxima semana integramos

Semana 15: navegación + componentes + hooks + servicios



Síntesis: Fetch, Axios y Alova son caminos para consumir REST; la calidad depende también de estado, errores, diagnóstico y separación.

Próxima semana: integración de proyectos React con navegación, componentes, hooks y servicios; PC3 según sílabo.



¡MUCHAS GRACIAS POR SU ATENCIÓN!

Universidad Tecnológica del Perú